

Artificial Intelligence Games: Homework 2

Name: Dai Xunlian
225040015

Homework-Game1: Branching Factor in Games

Context: Analyzing the board situation for the "red" player in Chinese Chess.

(A) How many different/distinct moves can he make? To determine the exact branching factor, we evaluate the legal moves for all active red pieces. Let's approximate based on standard opening configurations:

- **5 Pawns (Soldiers):** Assuming they have not crossed the river, they have 1 forward move each. Total = 5 moves.
- **2 Cannons:** As indicated in the slide examples, a cannon with open files can have up to 13 moves (e.g., $3 + 3 + 1 + 6 = 13$). If both are relatively open, they contribute approximately $13 \times 2 = 26$ moves.
- **Other Pieces (Horses, Rooks, Elephants, Advisors):** In a near-starting position, restricted pieces like Horses might have 2-4 moves, Rooks might have 2-3 moves depending on blocked paths.

Adding these up, a typical early-game branching factor b is approximately **40 to 44 distinct moves**.

(B) 1-step look-ahead strategy evaluation count: If the player uses a 1-step look-ahead, the algorithm evaluates every possible immediate next board state. Therefore, the number of heuristic evaluations is exactly equal to the branching factor b . **Answer:** Approximately **44 evaluations**.

(C) 3-step look-ahead strategy evaluation count: A 3-step look-ahead evaluates the tree at depth 3 (Red moves, Black moves, Red moves). Assuming an average branching factor $b \approx 44$ for both players, the number of leaf nodes evaluated is b^3 . **Answer:** $44^3 = 85,184$ evaluations.

Homework-Game2: Complete the Incomplete Tic-Tac-Toe Games

Using logical play, here are completions for the two incomplete games.

Game 1 sequence: 1. X center $\rightarrow$ 2. O top-left $\rightarrow$ 3. X right-center $\rightarrow$ 4. O bottom-right $\rightarrow$ 5. X must block at top-right to prevent O from getting a diagonal.

O		X
	X	X
		O

This path forces further blocks, ultimately resulting in a **Tie**.

Game 2 sequence: 1. X center $\rightarrow$ 2. O top-left $\rightarrow$ 3. X bottom-right $\rightarrow$ 4. O top-center $\rightarrow$ 5. X must block at top-right.

O	O	X
	X	
		X

Homework-Game3: Minimax Value Backup

To determine where to put the 1st X using value backup from the bottom layer:

1. **Terminal States:** The bottom layer consists of leaf nodes evaluated as +1 (X wins), -1 (O wins), or 0 (Tie).
2. **MIN Layer:** The layer above the leaves corresponds to O's turn. O will pick the minimum available value (favoring -1 or 0).
3. **MAX Layer:** X evaluates the layer above MIN, picking the maximum available value.
4. **Root Value:** If both players play optimally, Tic-Tac-Toe is a mathematically solved draw. Therefore, the value backup up to the root node yields exactly **0**.

Decision: Since the optimal outcome is 0 regardless of the first move (assuming the opponent is perfect), placing the 1st X in the **center** is the best heuristic move to maximize the opponent's chances of making an error, even though the backed-up minimax value is 0.

Homework-Game4: 2-Step-Ahead Search (Heuristic)

Using the heuristic $E(n) = M(n) - O(n)$ (where $M(n)$ is MAX's winning lines and $O(n)$ is MIN's winning lines):

- At the bottom of the 2-step look-ahead tree, we apply $E(n)$ to the resulting boards.
- The MIN layer selects the lowest heuristic value from its children to pass up.
- The MAX root node then selects the highest heuristic value among the options provided by the MIN layer.

Best Move: MAX should choose the branch that returns the highest backed-up value. If multiple branches return the same value, the move that inherently restricts O's central control (e.g., placing in a corner or center if available) looks better heuristically.

Homework-Game5: Tic-Tac-Toe Search Tree Properties

1. **Leaf nodes occur at levels 5, 6, 7, 8, 9. Why?**
A game of Tic-Tac-Toe requires a minimum of 5 moves to win (Player X makes 3 moves, Player O makes 2). Therefore, no game can end before level 5. The absolute maximum number of moves is 9 (the board is full), meaning all remaining paths must terminate by level 9.
2. **Leaf nodes on level 5 have a value of +1. Why?**
Level 5 is the result of X's 3rd move. Because O has only made 2 moves, O cannot possibly have 3-in-a-row. Thus, if the game ends at level 5, it is strictly because X has formed a winning line. The utility value for X winning is +1.
3. **Leaf nodes on level 6 have a value of -1. Why?**
Level 6 is the result of O's 3rd move. If the state is a leaf node at this level, it means the game just concluded on O's turn. Consequently, O must have won, yielding a utility value of -1.

4. Leaf nodes on level 9 have values 0 or 1. Why?

Level 9 is the final possible move of the game, made by X. The game ends either because X forms a winning line (yielding +1) or because the board is completely filled with no winner (yielding a Tie, 0). O cannot win on X's turn, so -1 is impossible here.

5. How many leaf nodes on level 5?

X has 8 possible winning lines. For X to win on move 5, X must place 3 pieces in one of these 8 lines. In the search tree (where permutation order matters), there are 1,440 specific sequences of play that result in a level 5 win for X.

6. How many leaf nodes on level 6?

Similar to X, O has 8 possible winning lines. Factoring in the permutations of X's 3 non-winning pieces and O's 3 winning pieces, there are 5,328 specific paths in the search tree that terminate at level 6.

7. How many leaf nodes on level 9?

This includes all games that end in a draw, plus all games where X wins specifically on the 9th move. Out of the $9! = 362,880$ total possible paths in the full tree, paths that reach level 9 without terminating earlier number in the tens of thousands.